Listen

Learning Doc 01: System Architecture Intuition

Written while designing the project, before any code exists

4 October 2026

Abstract

This document explains *how* and *why* the Listen system is shaped the way it is. Listen is a lightweight Windows voice assistant: it listens for commands, tracks which apps you use, carries out actions like "search what is an atom", and shows your analytics in a web app. You will not find code here. You will find the reasoning that comes *before* code, which is the part most tutorials skip and the part that matters most when a project grows.

Contents

1 What is "system architecture"? 2
- 1.1 Why decide before coding? . . . 2

2 Architecture starts from requirements 2
- 2.1 Listen's requirements . . . 2

3 The first big split: what *must* run where 3

4 The three candidate architectures 3
- 4.1 Option 1: Modular monolith + cloud hub (chosen) . . . 3
- 4.2 Option 2: Microservices with a message broker . . . 3
- 4.3 Option 3: Thin local agent, everything in the cloud . . . 4
- 4.4 Side-by-side . . . 4

5 The full picture 5

6 Inside the local agent 5
- 6.1 Modules and threads . . . 5
- 6.2 The event bus . . . 5
- 6.3 Pluggable pieces (one interface, many implementations) . . . 6
- 6.4 Keeping it lightweight . . . 6

7 Commands are data, not code 6

8 Data: local-first with selective sync 7
- 8.1 Two databases, two jobs . . . 7
- 8.2 What syncs and what stays . . . 8
- 8.3 Sync goes both ways . . . 8

9 The cloud side 8

10 Walkthrough: the life of one command 9

11 How projects are built (and where we are) **9**

12 Glossary **10**

13 Check your understanding **10**

1 What is "system architecture"?

Concept: System architecture

The **architecture** of a software system is the set of decisions about

1. **what the big pieces are** (components),
2. **where each piece runs** (your laptop? a server?),
3. **how the pieces talk to each other** (function calls? the internet?), and
4. **where data lives** and who is allowed to see it.

Architecture is about the decisions that are *expensive to change later*. The colour of a button is not architecture. Whether your audio is processed on your laptop or in the cloud *is*.

Analogy

Think of building a house. Before laying a single brick, an architect decides: how many floors, where the kitchen goes, where the plumbing runs. You can repaint a wall any time. Moving the plumbing after the house is built means tearing up floors. Software is the same: architecture is the plumbing.

1.1 Why decide before coding?

Beginners often start by writing code and "figuring it out as they go". That works for a 100-line script. For a project with several parts (a voice agent, a usage tracker, an API, a web app) it usually ends with a rewrite, because two parts were built on incompatible assumptions. Ten minutes of thinking on paper saves days of untangling code.

2 Architecture starts from requirements

You cannot judge an architecture as "good" in the abstract. It is only good *for a particular set of requirements*. Engineers split requirements into two kinds:

Concept: Functional vs. non-functional requirements

Functional requirements say *what the system does*: "when I say *search X*, open Google and search X".
Non-functional requirements say *how well / under what limits* it does it: "it must be lightweight", "my private conversations must not leave my laptop", "it must work offline".
Non-functional requirements shape the architecture the most. Almost any design can *eventually* open Google; far fewer can do it in 200 MB of RAM, offline, privately.

2.1 Listen's requirements

Functional:

- Listen through the microphone in three modes: wake word (default), continuous transcription (toggle), push-to-talk hotkey (fallback).
- Understand commands: `open`, `search`, `close`, `volume`, `play {query} on youtube`, with *default apps* when none is named.
- Let you add and edit commands *without touching code*, helped by an AI "command builder".
- Track which apps (and, from window titles, which websites) you use.
- Show analytics in a full web app; expose a documented API.

Non-functional (the ones that drive the design):

- **Lightweight**: runs all day on a 16 GB laptop with integrated graphics.
- **Private**: raw transcripts and raw window titles never leave the laptop.

- **Works offline**: commands still work and nothing is lost without internet.
- **Extensible**: new commands, speech engines and listening modes plug in.
- **Learnable**: this is a first project, so debuggability beats cleverness.

Keep this list in mind. Every decision below points back to one of these lines.

3 The first big split: what *must* run where

Some decisions are made for you by physics. A server in a data centre cannot hear your microphone, and it cannot see which window is open on your screen. So:

- The **voice agent** and **usage agent** *must* run on your laptop.
- The **web app** and **API** should run on a server, so you can open your dashboard from any device, and so they can be *deployed*.

Concept: Client and server

A **server** is a program that waits for requests and answers them (e.g. "give me my app-usage stats for this week"). A **client** is a program that sends those requests. The same machine can be both. In Listen, the laptop agent and the React web app are both *clients* of the cloud API.

This one observation already gives us the outline of the whole system: a **local side** and a **cloud side**, connected over the internet.

4 The three candidate architectures

Given "local side + cloud side", the open question was *how to organise the local side*. We compared three options.

4.1 Option 1: Modular monolith + cloud hub (chosen)

Concept: Monolith

A **monolith** is a single program (one process) that contains all the features. A **modular** monolith is still one program, but its inside is split into clearly separated modules that only talk through well-defined interfaces. You get the simplicity of one program with most of the organisation of separate services.

On the laptop, one Python process contains a voice module, a usage module, an action executor, a local database and a sync client. The modules run on separate *threads* and communicate through an in-memory *event bus* (Section 6).

4.2 Option 2: Microservices with a message broker

Concept: Microservices

Microservices split a system into many small, *separate programs*, each doing one job, talking over a network or a **message broker** (a program whose only job is passing messages between other programs, e.g. Redis, RabbitMQ).

The voice agent, usage agent and executor would each be their own Python program. If one crashes, the others keep running. But every Python program carries its own interpreter and libraries in memory (tens of MB each), plus the broker itself. That directly contradicts the *lightweight*

requirement. It is also much harder to debug: one bug might involve three programs and their message traffic. Microservices mostly solve an *organisational* problem: many teams working on one product without stepping on each other. We are one person.

4.3 Option 3: Thin local agent, everything in the cloud

The laptop would keep no data of its own and send every event straight to the cloud API. That is the simplest code, but it breaks *offline* (no internet means lost events and no command list) and *privacy* (everything leaves the laptop by default).

4.4 Side-by-side

	1. Modular monolith	**2. Microservices**	**3. Thin agent**
Lightweight	Best: one process	Worst: many processes + broker	Good
Works offline	Yes (local DB)	Yes	No
Privacy	Selective sync	Possible	Everything uploads
Debuggability	One terminal	Hard	Easy
Crash isolation	Medium (guarded modules)	Best	Medium
Can evolve later	Yes: modules can be split out	n/a	Needs redesign

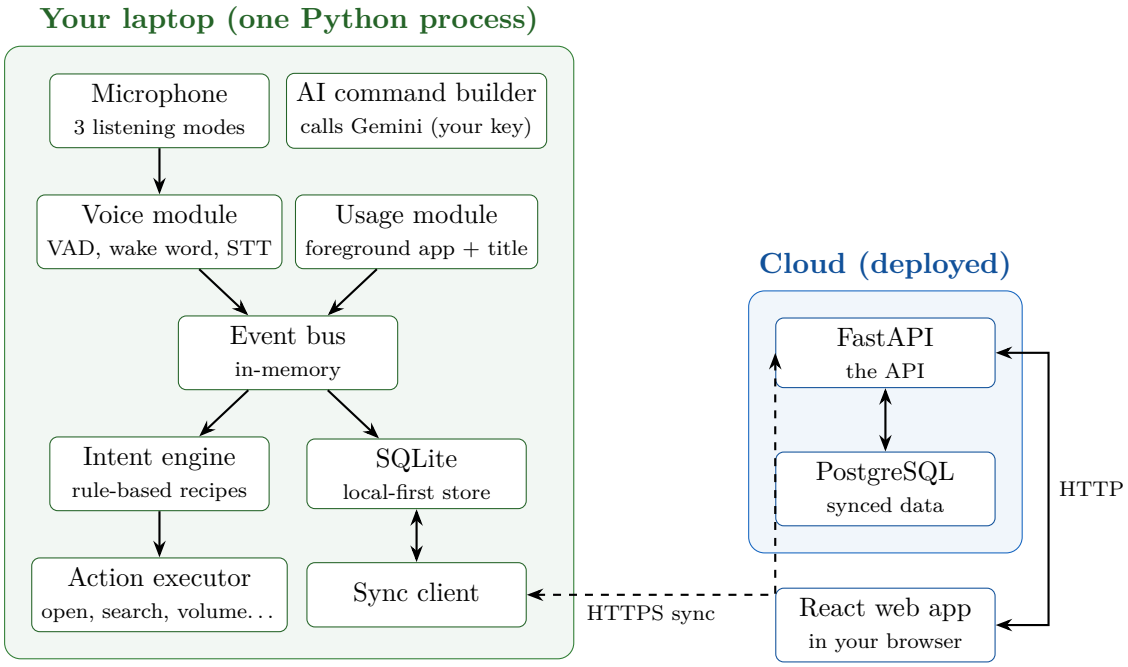
Why we chose this

Option 1 is the only one that satisfies *all three* hard requirements at once: lightweight, offline-capable and private. Its main weakness (one bad crash takes the whole agent down) is handled by wrapping each module in error handling, which is far cheaper than running a microservice fleet. And because the modules have clean boundaries, if one ever needs to become its own process, that is a small change instead of a rewrite.

Watch out

"Microservices" sounds more impressive, and many tutorials push it. Picking an architecture for how it sounds, instead of for your requirements, is one of the most common mistakes in software. Start with the simplest design that meets the requirements; split things later *only* when a real problem appears.

5 The full picture



How to read this diagram. Green is your laptop, blue is the cloud. Solid arrows are function calls inside one program; the dashed arrow is network traffic over the internet. The React app runs in your browser and talks *only* to the API over HTTP; it never touches the database directly.

6 Inside the local agent

6.1 Modules and threads

Concept: Thread

A **thread** is a sequence of instructions running inside a program. One program can have several threads, so the voice module can wait for audio *while* the usage module checks the foreground window every few seconds. Threads share the program's memory, so they are much lighter than separate programs.

6.2 The event bus

Concept: Event bus (publish / subscribe)

Instead of modules calling each other directly, a module **publishes** an *event* ("command heard: `search what is an atom`"), and any module that **subscribed** to that kind of event gets it. The publisher does not know or care who is listening.

Analogy

A radio station broadcasts; anyone with a radio tuned in hears it. The station does not keep a list of listeners. If you add a new listener (say, an "analytics logger"), the station does not change at all.

Why we chose this

Without a bus, the voice module would need to know about the intent engine, the database and the logger, and every new feature would mean editing the voice module. With a bus, new features just subscribe. This is called **loose coupling**: parts depend on a shared message format instead of on each other.

6.3 Pluggable pieces (one interface, many implementations)

You asked for three listening modes, two kinds of speech engine, and two ways to build commands. All of these follow one pattern:

Concept: Interface + interchangeable implementations

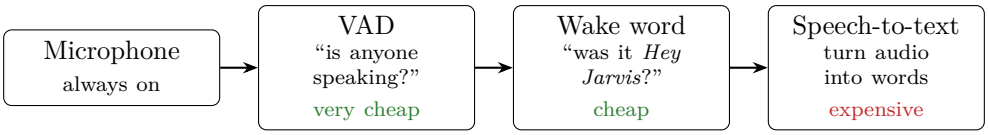
Define *what* a piece must do (its **interface**), for example "a `SpeechEngine` takes audio and returns text". Then write several pieces that all honour it: `FasterWhisperEngine`, `CloudEngine`. The rest of the program only talks to the interface, so swapping one engine for another is a setting, not a rewrite. (Textbooks call this the *strategy pattern*.)

Interface	Implementations in Listen
`AudioTrigger`	Wake word (default) · Continuous · Push-to-talk hotkey
`SpeechEngine`	faster-whisper (offline, default) · Cloud STT (optional)
`CommandBuilder`	LLM builder (online) · Guided form (offline fallback)
`Shell`	Terminal (v1) · System tray app (later)

Notice the last row: "terminal first, tray later" is not a rewrite. The tray is just a different *shell* around the same core.

6.4 Keeping it lightweight

The single most important trick is a **gate**: cheap work runs all the time, expensive work runs only when the cheap work says it is needed.



Concept: VAD (voice activity detection)

A tiny model that answers only one question: "is this sound a human voice?". Silence, fans and keyboard clicks are thrown away before anything expensive runs. In continuous mode the wake-word step is skipped, so VAD is what keeps continuous mode affordable.

The exact memory and CPU numbers depend on the models we pick, so we will *measure* them on your laptop during the build instead of trusting estimates.

7 Commands are data, not code

This is the idea that makes "add a command without touching code" possible.

The Python code knows only a small, fixed set of **action primitives**: `open_url`, `launch_app`, `close_app`, `set_volume`, `press_keys`. Every command is a *recipe* that combines a pattern with one of these primitives, stored in a data file:

```
- pattern: "search {query}"
  action: open_url
  target: "{default:search}"          # filled from your defaults

- pattern: "play {query} on spotify"
  action: open_url
  target: "https://open.spotify.com/search/{query}"
```

Analogy

A kitchen has a fixed set of appliances (oven, blender, stove): those are the primitives. Recipes are written on cards: those are the commands. Adding a new dish means writing a new card, not buying a new oven. The AI command builder is a helper who writes the card for you; you read it and approve it before it goes in the box.

Watch out

The limit: a command that needs a *new kind of action* (for example, controlling smart lights) needs a new primitive, and that is code. Anything built from existing primitives needs none.

Defaults. Each verb (`search`, `play`) has a default target. The usage module *suggests* defaults from your history ("you search on Google most"), and you can override them in settings. "`play lofi`" with no app named goes to YouTube because YouTube is the default for `play`.

8 Data: local-first with selective sync

Concept: Local-first

Every piece of data is written to a database *on your laptop* first. The internet is treated as optional: when it is available, data is synchronised; when it is not, nothing breaks and nothing is lost.

8.1 Two databases, two jobs

	SQLite (laptop)	**PostgreSQL (cloud)**
What it is	A database that is just one file; no server process	A full database server
Why here	Zero setup, almost zero memory: ideal for a lightweight agent	Handles many connections; standard for deployed APIs
Holds	Everything, including private data	Only what you chose to sync

8.2 What syncs and what stays

Data	Stays on laptop	Synced to cloud
Commands you gave and their results		✓
App-usage totals (“Chrome: 3h 12m”)		✓
Cleaned site names (“Google Search”)		✓
Raw continuous-mode transcripts	✓(auto-purged after N days)	
Raw window titles	✓(auto-purged after N days)	

8.3 Sync goes both ways

- **Up** (laptop → cloud): command log and usage summaries, every few minutes.
- **Down** (cloud → laptop): commands and settings you edited in the web app.

Concept: Source of truth

When the same data exists in two places, you must decide which copy wins if they disagree. For Listen: the **laptop** is the source of truth for *events* (what you said, what you used); the **cloud** is the source of truth for the *command library and settings*. Offline edits are queued and sent when the connection returns. The detailed conflict rules come in the data design step.

Why we chose this

This gives you a deployed dashboard with all the interesting analytics, while the most sensitive data (other people’s conversations, your email subjects in window titles) never leaves your machine. If the server were ever breached, the damage is limited by design.

9 The cloud side

- **FastAPI** (Python): the API. It receives sync data, serves analytics, and stores a copy of the command library for the web app. It generates its own interactive documentation, so the API is usable by others with no extra work.
- **PostgreSQL**: the cloud database.
- **React**: the web app (dashboard, command manager, settings). It is a separate program that only talks to the API over HTTP, exactly like any third-party client.

Concept: API-first

Build the API as if strangers will use it, then make your own web app its first customer. If your own app can do everything through the API, the API is genuinely complete, and multi-user support later (each person with their own laptop agent) becomes an extension, not a redesign.

Where the command builder lives (updated in Section 6). This first draft put the AI command builder in the cloud. It was later moved to the **laptop**: the API key is your own, pasted into the agent’s `.env`, and the free server then needs no paid service at all. The web app edits commands with a guided form instead. A good example of an early decision revisited once the requirements (here: a free-tier budget) became clearer.

Future-proofing, cheaply. Every stored record carries a `user_id` from day one, even though you are the only user. Adding that column now costs one line; adding it later means migrating every table. Rule of thumb: *pay small costs early only when they prevent a painful rewrite later.* Do not build the multi-user features themselves until you need them; that principle is called **YAGNI** ("You Aren't Gonna Need It").

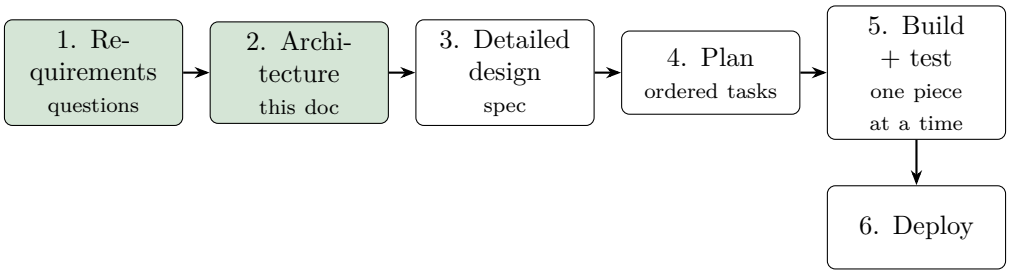
10 Walkthrough: the life of one command

You say: *"Hey Jarvis . . . search what is an atom"*.

1. **Microphone** streams audio into the voice module.
2. **VAD** notices human speech and passes those chunks on.
3. **Wake-word model** recognises "Hey Jarvis" and opens a recording window of a few seconds.
4. **Speech engine** (faster-whisper) turns that audio into the text `"search what is an atom"`.
5. The voice module **publishes** a `CommandHeard` event on the bus.
6. The **intent engine** matches it to the recipe `search {query}` with `query = "what is an atom"`. The target is the default for `search`: Google.
7. The **action executor** runs the primitive `open_url` with `https://www.google.com/search?q=what+is+an+atom`.
8. A `CommandExecuted` event is published; the **database** subscriber writes it to SQLite.
9. A few minutes later the **sync client** uploads it to the API, and it appears in your **dashboard**.

Every step is a separate, testable piece. When something goes wrong, the terminal shows exactly which step failed. That is the payoff of the architecture.

11 How projects are built (and where we are)



Green phases are done. Real teams loop back: building often reveals something the design missed, and that is normal. The goal of design is not to predict everything; it is to make the *expensive* decisions deliberately, so that the surprises are cheap ones.

Planned build order. Each stage produces something you can run and see:

1. Voice agent in the terminal (wake word → text → command → action).
2. Usage agent + local SQLite.
3. FastAPI + PostgreSQL, sync client.
4. React web app (dashboard, command manager, settings), AI command builder.
5. Tray app, then deployment.

12 Glossary

API A defined set of requests a program accepts, so other programs can use it.

Client / server
The program that asks / the program that answers.

Event bus
A shared channel where modules publish events and others subscribe.

Interface
A promise about what a component does, without saying how.

Local-first
Data lives on your device first; the network is optional.

Loose coupling
Parts depend on shared message formats, not on each other.

Microservices
A system made of many separate small programs.

Modular monolith
One program, internally split into well-separated modules.

Non-functional requirement
A quality limit: speed, memory, privacy, offline.

Primitive
One of the few basic actions the code knows how to perform.

Source of truth
The copy of the data that wins when copies disagree.

STT Speech-to-text.

Thread
A line of execution inside a program; several can run at once.

VAD Voice activity detection: "is this sound a human voice?".

YAGNI
"You Aren't Gonna Need It": do not build features before you need them.

13 Check your understanding

Check yourself

1. Why can't the voice agent simply run on the cloud server?
2. Name the non-functional requirement that ruled out microservices, and explain how.
3. You want to add "`play {query} on spotify`". Which parts of the system change? Which do not?
4. You want Listen to dim your smart lights. Why does this one need code?
5. Your Wi-Fi drops for an hour while you use Listen. What happens to the commands you give during that hour?
6. Why does the React app talk to the API instead of reading PostgreSQL directly?

7. What is the event bus buying us? Describe what would change in the voice module if we added a new "sound effect on command" feature, with and without the bus.

Next learning doc: the detailed design: the data model (tables and what goes in them), the sync protocol, and the command-recipe format.